

Beyond Executability: Hierarchical Feedback for Cross-Framework Training-Code Migration Repair

Anonymous Authors
Anonymous Institution
anonymous@example.com

Abstract—Cross-framework migration of deep learning training code must preserve more than executability: a translated program must reproduce forward values, gradients, and parameter updates across different runtime and automatic-differentiation systems. Existing translation and repair pipelines often rely on binary tests or execution errors, allowing a runnable candidate to silently change training semantics. We introduce a hierarchical feedback framework that compares paired executions through execution, forward-numerical, and gradient/update signals and reports the first failing semantic layer for targeted repair and strict re-verification. A common diagnostic schema separates this feedback logic from backend-specific execution, allowing the same design to operate on both `torch4ms`/MindSpore-backed and TorchAX/JAX-backed migration paths. On a 12-task sentinel set, execution-only feedback repairs 4/12 tasks, execution-plus-numerical feedback repairs 8/12, and the full hierarchy repairs 12/12 under the same repair budget. The final real-LLM repair loop succeeds on 46/50 instances across 25 migration-fault categories. Without changing the feedback hierarchy, the TorchAX/JAX instantiation correctly localizes and repairs all six execution, numerical, and gradient/update faults. These results demonstrate that hierarchical semantic feedback provides actionable repair guidance and transfers across framework backends.

Index Terms—cross-framework migration, training-code translation, hierarchical feedback, program repair, differential testing

I. INTRODUCTION

Large language models (LLMs) have made source-code translation increasingly practical, and recent systems improve generated programs through execution, repository context, and iterative feedback [1]–[5]. Cross-framework migration of deep learning (DL) training code raises a stricter problem. A migrated program must preserve not only control flow and API behavior, but also the tensor computations, automatic differentiation, optimizer state, and parameter updates that determine learning.

This requirement creates failures that ordinary executability checks cannot see. A translated training script may run successfully while producing different logits or losses. It may match the forward computation but propagate incorrect gradients, or compute plausible gradients while updating a different parameter set. Such failures arise across framework boundaries because runtimes differ in operator conventions, execution modes, differentiation interfaces, and optimizer semantics [6]–[8]. A successful run is therefore only the first condition of a correct training migration.

Existing translation and repair systems commonly reduce feedback to compilation, execution, unit-test outcomes, or coarse error messages [9]–[12]. These signals can identify a broken program, but they do not reveal where a runnable training program first diverges from the source semantics. Binary feedback consequently maps execution failures, forward mismatches, and backward/update errors to the same outcome even though they require different repairs.

Our central observation is that training correctness has an ordered semantic structure. A candidate must first execute, then preserve the forward computation, and finally preserve gradients and parameter updates. The first failing layer in this sequence provides an actionable diagnosis: execution failures point to compatibility and lowering, forward failures point to operator semantics, and gradient/update failures point to automatic differentiation, parameter registration, or optimizer behavior.

Based on this observation, we propose a hierarchical feedback mechanism with execution, forward-numerical, and gradient/update signals. Paired source and candidate executions are normalized into a common diagnosis report, and a repair component uses the first failing layer to produce a targeted patch before strict re-verification. The feedback schema is independent of a particular backend. We instantiate the same hierarchy on a `torch4ms`/MindSpore-backed path¹ and on a TorchAX/JAX-backed path without redesigning its semantic layers.

The experiments support the approach at the effect, mechanism, and transfer levels. Under the same repair budget, execution-only feedback repairs 4/12 sentinel tasks, adding forward-numerical feedback repairs 8/12, and the full hierarchy repairs 12/12. On the 50-instance fault pool, the final real-LLM repair loop succeeds on 46/50 instances spanning 25 fault categories. The TorchAX/JAX instantiation correctly localizes and repairs all six faults, demonstrating that the hierarchy transfers across framework backends.

This paper makes three contributions:

- We formulate cross-framework training-code migration as an ordered semantic validation problem and identify the first failing layer as the key signal for diagnosis-guided repair.

¹The primary prototype repository is available at <https://gitee.com/feixiao13/ascend-torch4ms.git>.

- We design a backend-independent hierarchy over execution, forward-numerical, and gradient/update behavior and integrate it with translation, targeted repair, and strict re-verification.
- We establish the effect and mechanism of hierarchical feedback on MindSpore-backed experiments and demonstrate cross-framework transfer through a second TorchAX/JAX-backed instantiation.

II. RELATED WORK

A. Code Translation and Validation

Early neural code translation showed that programming languages can be translated without direct supervision when models are trained on multilingual code corpora [13]. Later work improved this line by using automated unit tests as validation and self-training signals [14] and by introducing compiler representations to reduce source-target semantic gaps [15]. Recent LLM-based systems further incorporate executability representations, intermediate translation paths, repository-level context, and validation [1]–[3]. These studies motivate feedback-aware translation pipelines. Their validation, however, is typically organized around compilation, unit tests, or repository-level pass rates. DL framework migration needs a more specific validator that distinguishes execution failures, forward numerical divergence, and backward/update inconsistency in training code.

B. Feedback-Driven Repair and Multi-Agent Systems

Automated program repair has long used tests or semantic constraints to guide patch generation [16]. Large pre-trained language models have changed this space by making patch generation more fluent and enabling iterative repair from execution feedback [12]. Reflection and agent-based methods feed natural-language feedback, self-debugging traces, or executable code actions back into the generation loop [9]–[11]. Multi-agent LLM systems further show that role specialization, structured communication, shared artifacts, and explicit workflows can improve complex software tasks [17]–[20]. These approaches motivate our translator–verifier–fixer decomposition; our verifier contributes the migration-specific semantic layers that connect agent feedback to targeted training-code repair.

C. Framework Migration and Runtime Bridging

Modern DL frameworks expose different execution models and optimization abstractions. PyTorch emphasizes an imperative, dynamic front end [6], whereas TensorFlow demonstrates large-scale dataflow execution for machine learning systems [21]; compiler infrastructures such as MLIR further show how domain-specific computation can be lowered through extensible intermediate representations [22]. Existing migration and bridging tools address parts of this compatibility problem. MindConverter targets migration from third-party frameworks to MindSpore [23]. Runtime bridges such as TorchAX preserve a PyTorch-style front end while dispatching execution to another backend [8]. Our hierarchy builds

on these migration paths by diagnosing preservation at the execution, forward, gradient, and update levels.

D. Differential and DL-System Testing

Differential testing has been effective for finding compiler and program-analysis bugs by comparing semantically related executions [24], [25]. Testing work for DL systems has similarly explored behavioral coverage and differential-style validation of neural networks [26], [27]. These methods establish behavioral discrepancy as useful diagnostic evidence. Our work specializes differential validation to DL training migration and uses it to build hierarchical feedback over execution status, forward numerical behavior, gradient behavior, and parameter updates.

III. PROBLEM FORMULATION

A. Task Formulation

Given a training program P in a source framework F_s and a target framework path F_t , the migration task is to produce a candidate $\hat{P}$ that executes under F_t while preserving the training semantics of P . Training semantics includes the quantities that determine whether learning remains equivalent in practice: compatible tensors and modules, comparable forward values and losses, comparable gradients, and parameter updates with comparable direction and magnitude. Our primary instantiation migrates PyTorch training programs to a torch4ms/MindSpore-backed path, and the transfer study instantiates the same formulation on a TorchAX/JAX-backed path.

We evaluate correctness through paired executions under controlled inputs, synchronized initialization, and matched optimizer settings. Let $\mathcal{O}_t(P)$ denote the observable state at training step t : execution status, selected outputs or losses, gradients, and parameter-update statistics. A candidate is semantically aligned when $\mathcal{O}_t(\hat{P})$ remains within predefined tolerances of $\mathcal{O}_t(P)$ for the checked steps. The formulation focuses on framework-independent training observables, allowing the same diagnostic logic to compare executions produced by different target runtimes.

B. Diagnosis-Guided Repair Objective

The repair objective is not merely to assign a final pass/fail label. It is to identify the earliest semantic layer at which $\hat{P}$ departs from P and expose evidence that supports a corresponding edit. Execution failures call for compatibility or lowering changes; forward mismatches call for operator-semantics changes; and gradient/update mismatches call for differentiation, parameter-registration, or optimizer changes. A useful verifier must therefore preserve this diagnostic structure in the feedback delivered to the repair component.

IV. HIERARCHICAL SEMANTIC FEEDBACK

A. Design Principle

Training-code correctness is evaluated as an ordered hierarchy. A migrated program must first execute under the target backend, then preserve the forward computation, and finally

preserve the backward and update behavior that determines learning. We therefore organize validation around three semantic layers:

execution $\rightarrow$ forward numerical $\rightarrow$ gradient/update.

This ordering defines both a testing procedure and a repair policy. A deeper comparison is interpreted only after its prerequisites are available, and the first failing layer becomes the primary diagnosis. Later signals may provide supporting evidence, but they do not replace the earliest point at which the source and candidate executions diverge.

B. Framework Adapter Interface

The hierarchy operates on a common observation schema rather than framework-specific error formats. Each target path supplies an execution adapter that runs the candidate and exposes phase status, selected forward values, gradients, and parameter deltas in a normalized report. Runtime-specific details remain inside the adapter; the layer ordering, comparison metrics, diagnosis format, and repair routing remain unchanged. The `torch4ms/MindSpore` and `TorchAX/JAX` instantiations use this same interface, which makes cross-framework transfer a property of the design rather than a separate repair strategy.

C. Execution Signal

The execution signal checks whether $\hat{P}$ can complete the required training step under the target backend. It records whether module construction, data movement, forward execution, loss computation, backward execution, and optimizer stepping terminate without runtime failure. When a failure occurs, the verifier stores the exception type, stack location, operator or API name when available, tensor shape and dtype context, and the phase in which the failure appears. Typical failure classes include missing operators, unsupported lowering paths, interface mismatches, device-placement errors, dtype incompatibilities, shape errors, and invalid optimizer or parameter-registration behavior.

D. Forward-Numerical Signal

The forward-numerical signal checks whether the computation of $\hat{P}$ remains aligned with P once both executions complete. Under controlled inputs (x, y) and synchronized initialization, the verifier compares quantities such as logits, selected intermediate activations, and losses. The minimal scalar comparison is

$$\Delta_{\text{loss}} = \left| L(P; x, y) - L(\hat{P}; x, y) \right|. \quad (1)$$

When intermediate outputs are available, the verifier also computes output- or layer-level discrepancies and locates the earliest module whose difference exceeds tolerance. This localization is important because an end-to-end loss mismatch can be caused by an early operator difference, a reduction convention, a broadcasting rule, a normalization layer, or a later loss-function discrepancy.

E. Gradient and Update Signal

The gradient/update signal checks whether the migrated candidate preserves backward and optimizer behavior. This layer is necessary because two programs may produce similar forward losses while computing different gradients, updating different parameter subsets, or applying optimizer state differently. The verifier compares gradient norms and parameter-update statistics:

$$\Delta_{\text{grad}} = \left| \|\nabla_{\theta} L(P)\|_2 - \|\nabla_{\hat{\theta}} L(\hat{P})\|_2 \right|, \quad (2)$$

$$\Delta_{\text{param}} = \frac{\|\Delta\theta - \Delta\hat{\theta}\|_2}{\|\Delta\theta\|_2 + \epsilon}. \quad (3)$$

In addition to aggregate norms, the verifier can record missing gradients, zero-gradient parameters, parameter-name mismatches, and update outliers. A normal forward loss with missing gradients suggests an automatic-differentiation or parameter-registration issue, whereas aligned gradients with divergent updates suggests an optimizer-state or parameter-step issue.

F. Structured Diagnosis and Repair Routing

The verifier emits the status of each layer, the first failing layer, the supporting metrics, and a bounded set of root-cause hints. The repair component routes execution failures to compatibility edits, forward-numerical failures to operator-semantics edits, and gradient/update failures to backward or optimizer edits. This report turns the hierarchy into an actionable interface between validation and repair instead of a collection of independent metrics.

V. CROSS-FRAMEWORK MIGRATION REPAIR SYSTEM

Figure 1 shows the primary instantiation as an iterative repair system rather than a single-pass converter. A translator produces an initial target-path candidate, the verifier compares paired source and candidate executions, and the fixer applies bounded patches from the structured diagnosis. An orchestrator controls rounds, stopping rules, rollback, and snapshot logging. The components communicate through a shared artifact pool containing the source program, candidate program, diagnosis report, and patch log, so every repair action is tied to an executable artifact and explicit evidence.

The primary execution adapter uses `torch4ms`, which provides a PyTorch-style front end and dispatches operators, backward execution, and optimizer behavior to `MindSpore`-backed implementations. The transfer instantiation uses `TorchAX` to execute through a JAX-backed target path. Both adapters expose the same phase status, forward values, gradient evidence, and parameter-update statistics to the verifier. The repair loop therefore changes its framework-specific execution layer while preserving the diagnostic hierarchy and diagnosis-to-repair policy.

For each migration anchor, paired harnesses synchronize inputs, initialization, and optimizer settings before recording execution status, loss and optional layer-level differences, gradient-norm differences, and parameter-update relative ℓ_2

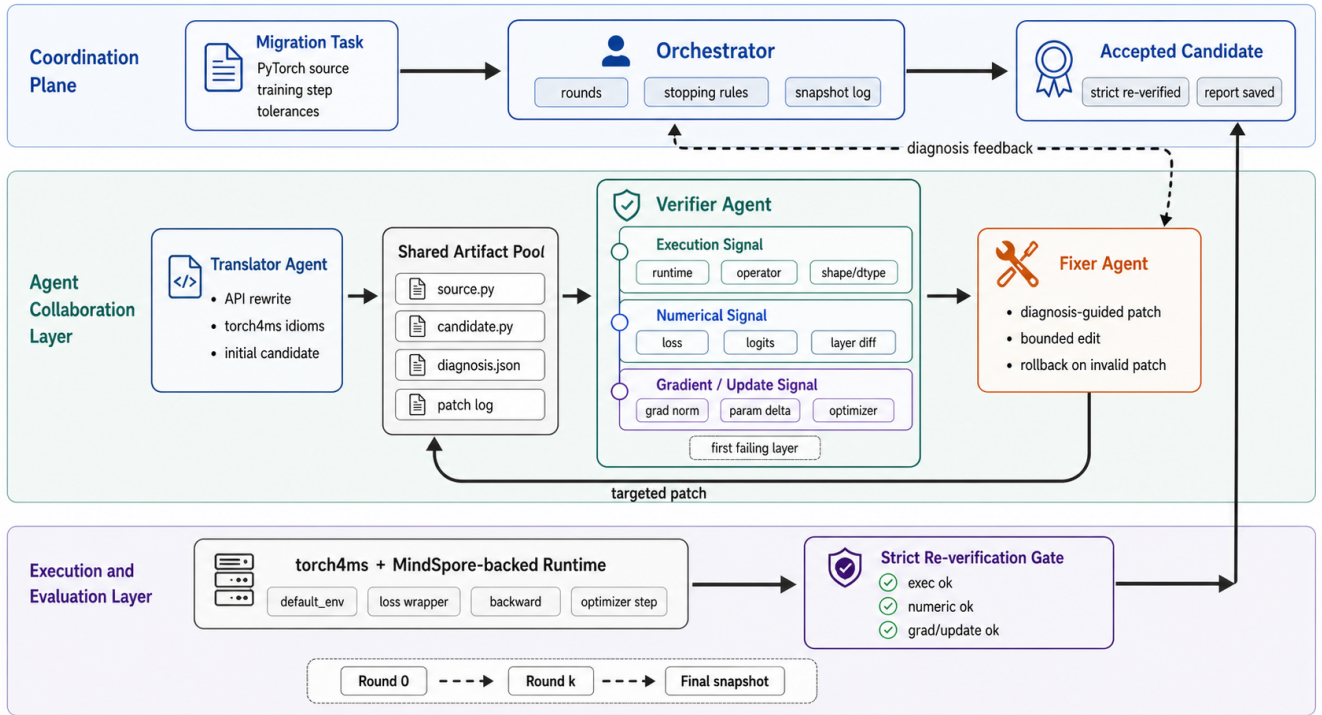


Fig. 1. Primary torch4ms/MindSpore-backed instantiation of the migration-repair system. Translation, hierarchical verification, diagnosis-guided repair, and strict re-verification communicate through explicit artifacts. The same execution, forward-numerical, and gradient/update diagnosis schema is reused by the TorchAX/JAX instantiation.

differences. A candidate is accepted only after the final snapshot passes strict re-verification across all configured semantic layers.

VI. EVALUATION

A. Research Questions

We organize the evaluation around three questions that follow the paper’s main claim:

- **RQ1: Repair effect.** Does hierarchical feedback improve strict repair success under the same repair budget?
- **RQ2: Diagnostic mechanism.** Do the semantic layers expose the intended failure types and support repair across failure layers and edit scopes?
- **RQ3: Cross-framework transfer.** Does the same hierarchy localize and repair faults on a second framework path without redesigning the feedback schema?

B. Experimental Scope and Metrics

The evaluation uses vision and language training programs, controlled migration faults, and real-LLM translation and repair. The MindSpore-backed experiments span CNN, MLP, Transformer, and tiny causal-LM families, three semantic failure layers, and candidate, core-operator, and core-autograd/optimizer edit scopes. The cross-framework experiment instantiates the same hierarchy on a TorchAX/JAX-backed path with CNN and MLP tasks and representative faults from all three semantic layers.

TABLE I
STRICT REPAIR SUCCESS UNDER MATCHED FEEDBACK CONDITIONS.

Feedback	Overall	Exec.	Num.	Grad./upd.
Exec. only	4/12	4/4	0/4	0/4
Exec.+Num.	8/12	4/4	4/4	0/4
Full	12/12	4/4	4/4	4/4

Each condition uses the same 12 tasks and four-round budget. Every successful repair finishes in one round, so Repair@1 and Repair@4 equal the displayed overall rates.

The main task-level metrics are strict repair success, Repair@ k , median repair rounds, and out-of-scope edit rate where applicable. Signal-level metrics include execution status, loss and optional layer-level differences, gradient-norm difference, and parameter-update relative ℓ_2 difference. Strict success requires the repaired artifact to satisfy the complete verification outcome defined for its experiment.

C. Feedback Mechanism Ablation

Design. To evaluate whether richer feedback improves repair under the same repair budget, we ran a real-LLM feedback-granularity ablation on four model families: CNN, image MLP, Transformer classifier, and tiny causal LM. Each model was evaluated under three sentinel fault categories: an execution fault (EX-01), a numerical fault (NU-04), and a gradient/update fault (GR-07). This balanced clean-alias sentinel set isolates the contribution of each semantic layer. The repair loop used one of three feedback conditions: execution-

TABLE II
SIGNAL EFFECTIVENESS UNDER CONTROLLED FAULTS.

Model	Fault	Det.	Signals
CNN	Exec.	1.000	E
CNN	Num.	1.000	N+G+U
CNN	Train.	1.000	G+U
MLP	Exec.	1.000	E
MLP	Num.	1.000	N+G+U
MLP	Train.	1.000	G+U
Trans.	Exec.	1.000	E
Trans.	Num.	1.000	N+G+U
Trans.	Train.	1.000	G+U
TinyLM	Exec.	1.000	E
TinyLM	Num.	1.000	N+U
TinyLM	Train.	1.000	G+U

Det. is expected-signal detection rate over three seeds (3/3 detections for every row). E, N, G, and U denote execution, numerical, gradient, and parameter-update signals.

only feedback, execution plus numerical feedback, or the full execution/numerical/gradient-update diagnosis. This produced 36 repair episodes in total. Every episode records final execution, numerical, and gradient/update pass fields, and strict success required all three recorded outcomes to pass.

Results. Table I shows a stepwise improvement as the repair loop receives the semantic layer needed for each fault class. Execution-only feedback repaired 4 of 12 tasks under strict verification, execution plus numerical feedback repaired 8 of 12, and the full feedback stack repaired all 12 tasks. Because every successful repair completed in the first round, Repair@1 and Repair@4 follow the same 33.3%, 66.7%, and 100.0% pattern. Runtime evidence repairs all four execution faults. Numerical diagnostics add all four numerical repairs, and gradient/update diagnostics add all four gradient/update repairs.

This controlled ablation establishes the causal role of the hierarchy. Execution feedback handles runtime failures, forward-numerical feedback exposes runnable programs with incorrect computation, and gradient/update feedback resolves failures that remain invisible after execution and loss checks. The broader evaluation in Section VI-D2 then measures the complete repair system over 25 fault categories.

D. Diagnostic Mechanism and Final Repair Capability

1) *Signal Localization under Controlled Faults: Design.* We evaluated whether each verifier signal detects the fault class it is designed to expose. This experiment uses controlled fault injection rather than automatic repair. For each model and seed, we compare a PyTorch reference step with a `torch4ms` candidate under one of three injected conditions: an execution fault, a numerical fault, or a training fault. The evaluation uses four model families (CNN, MLP, Transformer, and tiny causal LM), three seeds per fault, and thresholds of $\Delta_{\text{loss}} > 0.02$, $\Delta_{\text{grad}} > 0.05$, and $\Delta_{\text{param}} > 0.03$.

Results. Table II shows that the expected signal detects its corresponding fault in all 36 fault-injection runs. Execution faults are isolated by the execution signal. For training faults,

TABLE III
END-TO-END REPAIR RESULTS ON THE MINDSPORE FAULT POOL AND TORCHAX/JAX TRANSFER.

<i>(a) MindSpore fault pool by failure layer</i>			
Failure layer	Inst.	Strict succ.	Repair@1
Execution	20	19 (95.0%)	65.0%
Numerical	14	13 (92.9%)	50.0%
Gradient/update	16	14 (87.5%)	62.5%
Overall	50	46 (92.0%)	60.0%
<i>(b) MindSpore fault pool by repair scope</i>			
Repair scope	Inst.	Strict succ.	Repair@1
Candidate-level	26	25 (96.2%)	84.6%
Core operator	20	19 (95.0%)	40.0%
Core A/O	4	2 (50.0%)	0.0%
<i>(c) TorchAX/JAX cross-framework transfer</i>			
Fault type	Tasks	Layer loc.	Strict repair
Execution	2	2/2	2/2
Forward numerical	2	2/2	2/2
Gradient/update	2	2/2	2/2
Overall	6	6/6	6/6

A/O denotes autograd/optimizer. Full-pool strict success equals Repair@4. All TorchAX/JAX repairs finish in one round, so Repair@1 and Repair@3 are both 100.0%.

G records missing-gradient evidence and U captures the divergent parameter update. Numerical faults are detected at the forward layer even when their effects propagate to gradient or update metrics. The first-failure rule therefore preserves a distinct diagnostic handle for every fault class.

2) *Final Repair Evaluation on a Fixed Fault Pool: Design.* We next evaluated whether the recorded verifier diagnoses and real-LLM fixer could be turned into repairs. This experiment used a fixed pool of 25 migration-fault categories and 50 model-assigned instances. The pool covered four model families (CNN, image MLP, Transformer classifier, and tiny causal LM), three failure layers, and three repair scopes: candidate-level code, core operators, and core autograd/optimizer behavior. All 50 instances were hidden from the repair prompts. Each instance allowed at most four repair attempts. Strict success required the repaired artifact to pass the original fault oracle and strict re-verification; failures were restored or rolled back.

Results. Panels (a) and (b) of Table III show that the final repair loop achieved 46/50 strict successes, corresponding to an overall strict repair rate of 92.0%. The system repaired 19/20 execution instances, 13/14 numerical instances, and 14/16 gradient/update instances. By repair scope, it repaired 25/26 candidate-level and 19/20 core-operator instances; core autograd/optimizer repair reached 2/4. Every fault category had at least one repaired instance, yielding 100% category coverage. The four remaining cases comprise one invalid execution patch, one numerical instance that exhausted the repair budget, and two unresolved gradient/update mismatches.

E. Cross-Framework Transfer to TorchAX/JAX

Design. We instantiated the same execution, forward-numerical, and gradient/update hierarchy on a TorchAX/JAX-backed target path. The layer order, normalized diagnosis fields, first-failure rule, and repair routing were unchanged. The evaluation used MLP and CNN tasks with two exe-

TABLE IV
PLATFORM-GUIDANCE ABLATIONS FOR ITERATIVE REPAIR AND ONE-SHOT TRANSLATION.

(a) Fixer guidance by repair scope

Scope	Guide	Strict B/L	L R@1/4	L OOS
Candidate	Off	0/12; 12/12	100.0/100.0%	0.0%
Candidate	On	0/12; 8/12	58.3/66.7%	0.0%
Core operator	Off	0/8; 6/8	50.0/75.0%	12.5%
Core operator	On	0/8; 6/8	50.0/75.0%	0.0%
Core A/O	Off	0/4; 0/4	0.0/0.0%	0.0%
Core A/O	On	0/4; 0/4	0.0/0.0%	0.0%

(b) One-shot Translator guidance with the Fixer disabled

Condition	Runs	Compile	Execute	Train	Trans.@1
Guide off	15	13.3%	13.3%	0.0%	0.0%
Guide on	15	100.0%	100.0%	100.0%	100.0%

B/L denotes binary/layered strict success; R@1/4 and OOS are for layered feedback. A/O denotes autograd/optimizer. All binary-feedback R@1, R@4, and OOS rates are 0.0%.

cution, two numerical, and two gradient/update faults. For each fault, we first measured first-layer localization and then ran candidate-level repair with the same strict re-verification policy.

Results. Panel (c) of Table III shows complete transfer of the hierarchy to the TorchAX/JAX-backed path. The verifier localizes all six faults at the correct semantic layer, and every diagnosis leads to a candidate that passes strict re-verification in one round. The execution cases expose runtime failures, the forward-numerical cases expose output or loss divergence, and the gradient/update cases expose missing-gradient and parameter-update evidence. Achieving 6/6 localization and 6/6 strict repair without changing the feedback schema demonstrates cross-framework portability of the diagnosis-to-repair mechanism.

F. Auxiliary Component Analyses

1) *Fixer Guidance by Repair Scope:* **Design.** We crossed binary versus layered verifier feedback with the Fixer guide disabled versus enabled while freezing the Translator. The experiment contains 24 base tasks over CNN and MLP models: 12 candidate-level tasks, eight core-operator tasks, and four core autograd/optimizer tasks. Each task was run under all four conditions with a four-round budget, producing 96 condition runs.

Results. Panel (a) of Table IV shows that layered feedback repairs 18/24 tasks without the Fixer guide and 14/24 with it, while binary feedback repairs none under either guide setting. The hierarchy is therefore the dominant source of repair improvement. The guide reduces core-operator out-of-scope edits from 12.5% to 0.0%, while candidate-level strict success changes from 12/12 to 8/12 and core-operator success remains 6/8. Core autograd/optimizer repair is the hardest scope in every condition.

2) *One-Shot Translator Guidance:* **Design.** To isolate Translator guidance, we disabled the Fixer and made one Translator call per run. Five held-out migration tasks—Image MLP, CNN, ResNet, Transformer classifier, and tiny causal

TABLE V
ORACLE-CANDIDATE 50-STEP TRAINING CONSISTENCY.

Model	Pass	Mean ΔL	Max ΔL	Mean ΔG	Mean ΔU
CNN	1.000	4.570×10^{-7}	2.980×10^{-6}	1.510×10^{-8}	1.146×10^{-6}
NLP	1.000	6.167×10^{-7}	1.669×10^{-6}	1.441×10^{-8}	3.825×10^{-6}
Trans.	1.000	5.643×10^{-8}	2.384×10^{-7}	1.748×10^{-8}	7.913×10^{-7}
TinyLM	1.000	1.745×10^{-6}	3.815×10^{-6}	3.835×10^{-8}	1.127×10^{-6}

Each row aggregates three seeds and 50 paired steps. ΔL , ΔG , and ΔU denote loss, gradient-norm, and parameter-update differences.

LM—were evaluated over three repeats in each condition, giving 15 runs without the guide and 15 with it.

Results. Panel (b) of Table IV shows that without Translator guidance, 2/15 outputs compile and execute, and none complete the training check. With guidance, all 15 outputs pass compilation, execution, training, and Translation Success@1. Together with the Fixer ablation, this result separates the strong effect of platform knowledge during initial translation from its different effect during iterative repair.

G. Training-Path Validation

Design. We use known-good oracle candidates to verify that the paired execution paths preserve short-horizon training behavior before interpreting repair outcomes. The evaluation contains CNN, NLP, Transformer, and tiny causal-LM trajectories, each run for 50 steps over three seeds. The per-step thresholds are $\Delta_{\text{loss}} \leq 0.02$, $\Delta_{\text{grad}} \leq 0.05$, and $\Delta_{\text{param}} \leq 0.03$.

Results. All four model trajectories pass the consistency check. The largest maximum loss difference is 3.815×10^{-6} , mean gradient-norm differences remain below 4×10^{-8} , and mean parameter-update relative ℓ_2 differences remain below 3.9×10^{-6} . These measurements validate the short-horizon execution paths used by the semantic verifier.

H. Limitations and Threats to Validity

The evaluation covers two target backend paths, four model families, 25 fault categories, three repair scopes, and 50-step training trajectories. Repository-scale programs, longer training runs, and additional framework pairs will test the hierarchy at larger scale. Complete per-step gradient capture and versioned prompt, verifier, dataset, and batch manifests will strengthen future replications.

VII. CONCLUSION

Cross-framework training-code migration requires feedback that reaches beyond executability. We introduced an ordered hierarchy over execution, forward-numerical, and gradient/update semantics and used the first failing layer to connect validation with targeted repair. The MindSpore-backed experiments show a stepwise repair gain as semantic feedback is added and a 46/50 strict success rate on the final fault pool. Reusing the same hierarchy on a TorchAX/JAX-backed path yields 6/6 first-layer localization and 6/6 strict repair, demonstrating cross-framework transfer without redesigning the diagnosis schema. These results establish hierarchical

semantic feedback as a practical foundation for training-code migration repair. Future work will broaden the framework pairs and program scale, strengthen complete gradient/update collection, and evaluate longer generated training trajectories.

REFERENCES

- [1] M. He, Y. Chen, F. Yang, P. Zhao, W. Yin, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang, “ExeCoder: Empowering large language models with executability representation for code translation,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Suzhou, China: Association for Computational Linguistics, 2025, pp. 7099–7125. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.362/>
- [2] M. Macedo, Y. Tian, P. Nie, F. R. Cogo, and B. Adams, “InterTrans: Leveraging transitive intermediate translations to enhance LLM-based code translation,” in *Proceedings of the 47th International Conference on Software Engineering*. IEEE, 2025, pp. 2819–2831. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00236>
- [3] Z. Guan, X. Yin, Z. Peng, and C. Ni, “RepoTransAgent: Multi-agent LLM framework for repository-aware code translation,” *arXiv preprint arXiv:2508.17720*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.17720>
- [4] E. Erer, A. Bozanta, T. Aytac, and A. Basar, “A fully automated agent for end-to-end code translation and validation,” in *Proceedings of the 2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2025, pp. 324–330. [Online]. Available: <https://doi.org/10.1109/ESEM64174.2025.00058>
- [5] H. Gray, T. Yotam, and O. Udrea, “Workflows vs agents for code translation,” *arXiv preprint arXiv:2512.14762*, 2025, neurIPS 2025 Fourth Workshop on Deep Learning for Code. [Online]. Available: <https://arxiv.org/abs/2512.14762>
- [6] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: <https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [7] MindSpore, “MindSpore: A new deep learning computing framework,” <https://www.mindspore.cn/en>, 2020, framework white paper and documentation.
- [8] Google, “torchax: A PyTorch frontend for JAX,” <https://google.github.io/torchax/>, 2025, accessed: 2026-06-02.
- [9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.11366>
- [10] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching large language models to self-debug,” in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=KuPixIqPiq>
- [11] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. Vienna, Austria: PMLR, 2024, pp. 50 733–50 749. [Online]. Available: <https://proceedings.mlr.press/v235/wang24h.html>
- [12] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proceedings of the 45th International Conference on Software Engineering*. IEEE, 2023, pp. 1482–1494. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00129>
- [13] M.-A. Lachaux, B. Rozière, L. Chausson, and G. Lample, “Unsupervised translation of programming languages,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 20 601–20 611. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html>
- [14] B. Rozière, J. M. Zhang, F. Charton, M. Harman, G. Synnaeve, and G. Lample, “Leveraging automated unit tests for unsupervised code translation,” in *Proceedings of the Tenth International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=cmt-6KtR4c4>
- [15] M. Szafraniec, B. Rozière, H. J. Leather, P. Labatut, F. Charton, and G. Synnaeve, “Code translation with compiler representations,” in *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=XomEU3eNeSQ>
- [16] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, “SemFix: Program repair via semantic analysis,” in *Proceedings of the 35th International Conference on Software Engineering*. IEEE, 2013, pp. 772–781. [Online]. Available: <https://doi.org/10.1109/ICSE.2013.6606623>
- [17] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “ChatDev: Communicative agents for software development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 15 174–15 186. [Online]. Available: <https://aclanthology.org/2024.acl-long.810/>
- [18] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [19] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *Proceedings of the First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=uAjxFFing2>
- [20] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, “AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors,” in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=EHg5GDnyq1>
- [21] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: A system for large-scale machine learning,” in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 2016, pp. 265–283. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [22] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, “MLIR: Scaling compiler infrastructure for domain specific computation,” in *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization*, 2021, pp. 2–14. [Online]. Available: <https://doi.org/10.1109/CGO51591.2021.9370308>
- [23] MindSpore, “Migrating from third party frameworks with Mind-Converter,” https://www.mindspore.cn/mindinsight/docs/en/r2.0.0-alpha/migrate_3rd_scripts_mindconverter.html, 2023, accessed: 2026-06-02.
- [24] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2011, pp. 283–294. [Online]. Available: <https://doi.org/10.1145/1993498.1993532>
- [25] V. Le, M. Afshari, and Z. Su, “Compiler validation via equivalence modulo inputs,” in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2014, pp. 216–226. [Online]. Available: <https://doi.org/10.1145/2594291.2594334>
- [26] K. Pei, Y. Cao, J. Yang, and S. Jana, “DeepXplore: Automated whitebox testing of deep learning systems,” in *Proceedings of the 26th Symposium on Operating Systems Principles*. ACM, 2017, pp. 1–18. [Online]. Available: <https://doi.org/10.1145/3132747.3132785>
- [27] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu, J. Zhao, and Y. Wang, “DeepGauge: Multi-granularity testing criteria for deep learning systems,” in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. ACM, 2018, pp. 120–131. [Online]. Available: <https://doi.org/10.1145/3238147.3238202>